ASSIGNMENT 3

NAME: JANANIPRIYA G
REG.NO: 192324041
SUBMISSION DATE:12/11/26
SLOT: A
SUBJECT:JAVA PROGRAMMING
COURSE CODE: CSA0903

Q1 – HANDLING FAILURE IN A BANK ATM SIMULATION

1.1 Exception Hierarchy

Java exception hierarchy:

```text
Throwable
  |
  |-- Error
  |
  |-- Exception
        |
        |-- IOException (Checked)
        |
        |-- RuntimeException (Unchecked)
              |
              |-- InputMismatchException
              |-- ArithmeticException
              |-- NumberFormatException
```

```

1.2 Failure Points and Exception Types

| Failure Point | Exception | Type |

|---|---|---|

| Entering letters instead of numbers | InputMismatchException | Unchecked |

| Missing card-reader file | FileNotFoundException | Checked |

| Negative withdrawal amount | IllegalArgumentException | Unchecked |

| Withdrawal exceeds balance | InsufficientBalanceException | User-defined |

| Invalid account number | NumberFormatException | Unchecked |

1.3 Catch Block Order

Specific exceptions must be caught before general exceptions. For example, FileNotFoundException must appear before IOException, and IOException must appear before Exception.

If a general catch block is placed first, the compiler reports an unreachable catch block error.

1.4 Role of Finally Block

The finally block executes whether an exception occurs or not in normal control flow. It is used for cleanup operations such as closing files and displaying transaction completion messages.

1.5 Test Cases
```

| Test Input | Expected Result |

|---|---|

| Valid account and amount | Withdrawal successful |

| Letters instead of amount | InputMismatchException |

| Negative amount | Invalid withdrawal amount |

| Amount greater than balance | Insufficient balance |

| Missing card file | FileNotFoundException |

1.6 Joint Design Question

Catching every exception using `catch(Exception e)` is acceptable as a final safety net or for logging unexpected failures. However, it hides specific error details and makes debugging difficult. Therefore, specific exceptions should be handled separately.

1.7 Alternative Concept and Complexity Analysis

Instead of exceptions, input can be validated using `Scanner.hasNextInt()` and error codes such as -1.

- Exception handling provides clear error information.

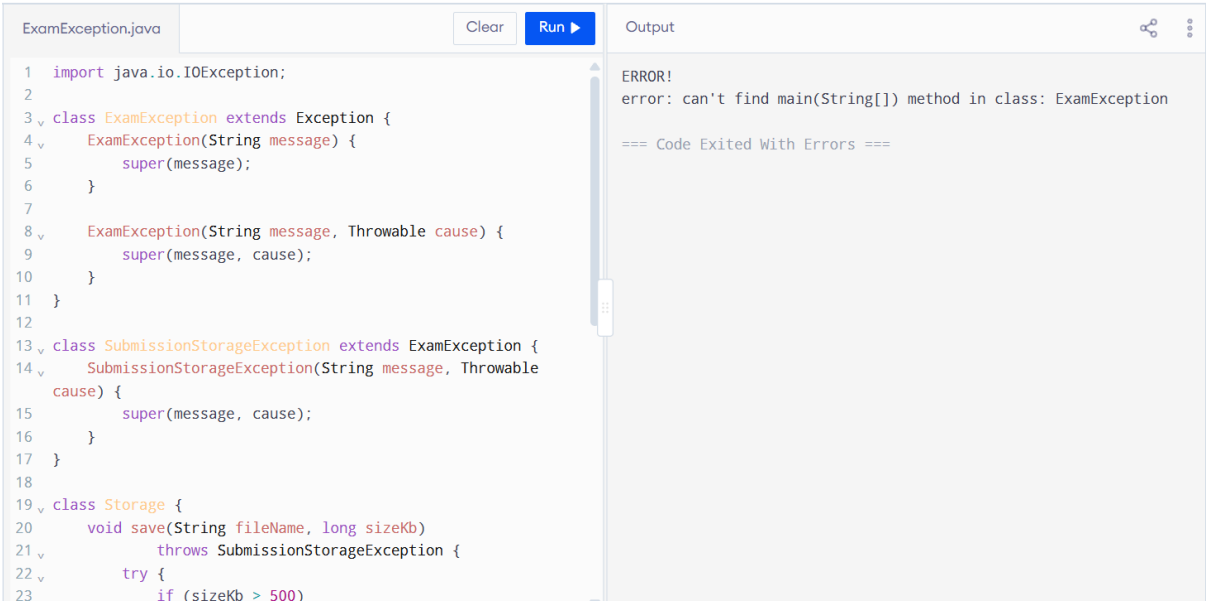
- Error codes avoid exception creation and stack-trace processing.

- Both approaches generally require O(1) time per validation and O(1) additional space, excluding exception stack-trace storage.

1.8 Conclusion

Exception handling makes the ATM program reliable by managing invalid inputs, insufficient balance and file errors. Specific catch blocks and finally ensure proper error handling and cleanup.

OUTPUT:



Q2 – USER-DEFINED EXCEPTIONS AND PROPAGATION IN AN ONLINE EXAMINATION PORTAL

2.1 Problem Analysis

An online examination portal must validate student submissions before accepting them. A submission may fail due to an expired exam time, an unregistered student or an oversized answer file.

User-defined exceptions help handle these errors clearly.

2.2 Exception Hierarchy

Exception

|

|-- ExamException

|

|-- ExamWindowClosedException

|

|-- StudentNotRegisteredException

|

|-- SubmissionStorageException

All these exceptions are checked exceptions because they extend Exception.

2.3 Exception Propagation

Exception propagation transfers an exception from one method to another until it is handled.

Example:

1. Storage layer detects a file-saving error.
2. Service layer receives the exception.
3. Controller layer catches and handles the exception.
4. A suitable error message is displayed to the student.

2.4 Java Implementation – Exception Chaining

```
import java.io.IOException;


class ExamException extends Exception {
  ExamException(String message) {
    super(message);
  }

  ExamException(String message, Throwable cause) {
    super(message, cause);
  }
}


class SubmissionStorageException extends ExamException {
  SubmissionStorageException(String message, Throwable cause) {
    super(message, cause);
  }
}
```

```
class Storage {
   void save(String fileName, long sizeKb)
         throws SubmissionStorageException {
      try {
         if (sizeKb > 500)
            throw new IOException("Disk quota exceeded");

         System.out.println("Saved " + fileName);

      } catch (IOException e) {
         throw new SubmissionStorageException(
            "Could not store " + fileName, e);
      }
   }
}

public class StorageDemo {
   public static void main(String[] args) {
      Storage storage = new Storage();

      try {
         storage.save("answer01.pdf", 120);
         storage.save("answer02.pdf", 800);

      } catch (ExamException e) {
         System.out.println("Error: " + e.getMessage());
         System.out.println("Cause: " + e.getCause().getMessage());
      }
```

```
    }
}
```

2.5 Expected Output

Saved answer01.pdf

Error: Could not store answer02.pdf

Cause: Disk quota exceeded

2.6 Test Scenarios

Scenario	Expected Result
Exam time expired	ExamWindowClosedException
Student not registered	StudentNotRegisteredException
File size exceeds 500 KB	SubmissionStorageException
Valid file submission	File saved successfully

2.7 Importance of Exception Chaining

Exception chaining preserves the original cause of an error using getCause().

If the IOException is not passed to the constructor, the Cause line prints null instead of Disk quota exceeded.

The storage layer should throw a meaningful SubmissionStorageException because it provides understandable application-level information without exposing internal storage details directly to the user interface.

2.8 Alternative Concept and Complexity Analysis

If custom exceptions extend RuntimeException instead of Exception:

- The exceptions become unchecked.
- The methods do not need to declare them using throws.
- The calling method is not forced to catch them.
- Error handling becomes optional for the caller.

Concept	Checked Exception	Unchecked Exception
Parent class	Exception	RuntimeException

Must declare or catch	Yes	No
Compiler checking	Yes	No

Creating and throwing exceptions requires additional time for object creation and stack-trace generation. Exception chaining also retains a reference to the original cause, increasing memory usage.

2.9 Conclusion

User-defined exceptions and exception chaining improve error handling in an online examination portal. They preserve the original cause of errors and provide meaningful messages to users.

Q3 – RESOURCE SAFETY, DEBUGGING AND ROBUST CODE

3.1 Problem Analysis

A retail chain generates daily sales reports by reading multiple store files and writing a summary file. If files are not closed properly, the program may produce a "Too many open files" error.

Resource management is important to prevent file leaks and incomplete reports.

3.2 Resources Used

The following resources must be closed after processing:

1. Input file readers.
2. Output file writers.
3. Buffered readers and writers.

3.3 Manual Resource Management

In manual resource management, files are closed using the close() method inside a finally block.

```
import java.io.*;


public class ManualFile {
  public static void main(String[] args) {
    BufferedReader br = null;


    try {
```

```
            br = new BufferedReader(new FileReader("sales.txt"));

            String line;
            while ((line = br.readLine()) != null) {
                System.out.println(line);
            }

        } catch (IOException e) {
            System.out.println("File error: " + e.getMessage());

        } finally {
            try {
                if (br != null)
                    br.close();
            } catch (IOException e) {
                System.out.println("Closing error");
            }
        }
    }
}
```

3.4 Try-with-Resources

Try-with-resources automatically closes resources after the try block finishes, even when an exception occurs.

```
import java.io.*;

public class TryResource {
    public static void main(String[] args) {
        try (BufferedReader br =
```

```
        new BufferedReader(new FileReader("sales.txt"))) {

        String line;
        while ((line = br.readLine()) != null) {
            System.out.println(line);
        }

    } catch (IOException e) {
        System.out.println("File error: " + e.getMessage());
    }
  }
}
```

3.5 Comparison

Manual Closing	Try-with-Resources
Uses close() manually	Closes automatically
Requires finally block	No finally block required for closing
More code	Less code
Greater chance of resource leaks	Safer resource management

3.6 Debugging and Experimental Analysis

Student A can use breakpoints and step-through debugging to identify where the manual version fails to close a file.

A test can be performed using multiple input files, including a missing or corrupted file.

Test Case	Manual Version	Try-with-Resources
All files available	Processes files	Processes files
Missing input file	Handles IOException	Handles IOException
Exception during processing	May leak if closing is incorrect	Automatically closes resources

Multiple files	Requires manual closing	Automatically closes resources

3.7 Suppressed Exception

A suppressed exception occurs when an exception happens while another exception is already being handled. In try-with-resources, a closing exception may be suppressed when the try block has already thrown an exception. It can be accessed using getSuppressed().

3.8 Joint Engineering Decision

Try-with-resources is not sufficient by itself when the program has incorrect file paths, corrupted data, insufficient permissions or incomplete report-writing logic. Proper validation and exception handling are also required.

3.9 Alternative Concept and Complexity Analysis

If try-with-resources is replaced with manual closing:

- Explicit close() calls must be written inside finally.
- Incorrect cleanup may cause file leaks.
- Without closing files, system resources remain occupied.

For f store files:

Operation	**Time Complexity**
Reading all files	O(f)
Processing file contents	O(n), where n is total records
Writing summary	O(n)

Try-with-resources and correct manual closing both use O(1) additional resource space when files are processed one at a time. Without closing files, the number of open file handles can grow up to O(f).

3.10 Conclusion

Try-with-resources is a safer and simpler method for managing files in Java. It automatically closes resources and reduces the possibility of file leaks and incomplete processing.

Q4 – THREAD CREATION AND THE THREAD LIFE CYCLE

4.1 Problem Analysis

An examination cell uses four scanner stations to scan answer scripts simultaneously. Each scanner station is represented by a separate thread that processes a batch of scripts.

Java supports two methods of creating threads:

1. Extending the Thread class.
2. Implementing the Runnable interface.

4.2 Creating Threads Using Thread Class

```
class ScannerStation extends Thread {
  String station;
  int scripts;

  ScannerStation(String station, int scripts) {
    this.station = station;
    this.scripts = scripts;
  }

  public void run() {
    System.out.println(station + " scanned " + scripts + " scripts");
  }
}

public class Main {
  public static void main(String[] args)
      throws InterruptedException {

    ScannerStation s1 = new ScannerStation("Station 1", 10);
    ScannerStation s2 = new ScannerStation("Station 2", 15);
    ScannerStation s3 = new ScannerStation("Station 3", 20);
    ScannerStation s4 = new ScannerStation("Station 4", 25);
```

```
        s1.start();
        s2.start();
        s3.start();
        s4.start();

        s1.join();
        s2.join();
        s3.join();
        s4.join();

        System.out.println("All stations completed");
    }
}
```

4.3 Creating Threads Using Runnable Interface

```
class ScannerTask implements Runnable {
    String station;
    int scripts;

    ScannerTask(String station, int scripts) {
        this.station = station;
        this.scripts = scripts;
    }

    public void run() {
        System.out.println(station + " scanned " + scripts + " scripts");
    }
}
```

```
public class RunnableDemo {
   public static void main(String[] args) {
      Thread t1 = new Thread(new ScannerTask("Station 1", 10));
      Thread t2 = new Thread(new ScannerTask("Station 2", 15));
      Thread t3 = new Thread(new ScannerTask("Station 3", 20));
      Thread t4 = new Thread(new ScannerTask("Station 4", 25));

      t1.start();
      t2.start();
      t3.start();
      t4.start();
   }
}
```

4.4 Difference Between Thread and Runnable

Thread	Runnable
Extends Thread class	Implements Runnable interface
Cannot extend another class	Can extend another class
Task and thread are combined	Task and thread are separated
Less flexible	More flexible

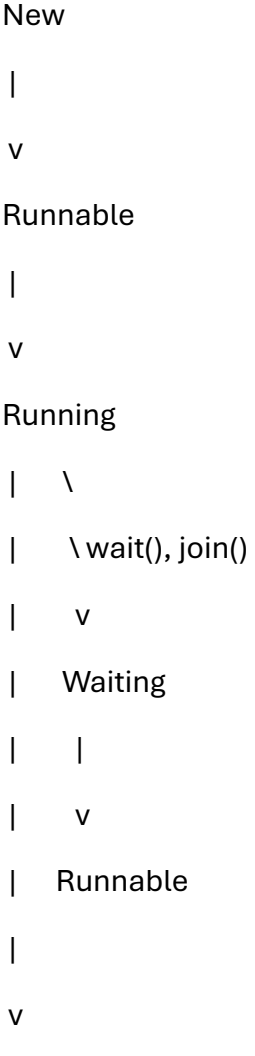
Runnable is generally preferred because it separates the task from the thread.

4.5 Thread Life Cycle

The main states of a Java thread are:

1. **New:** Thread object is created.
2. **Runnable:** Thread is ready to execute.
3. **Blocked:** Thread waits to acquire a lock.
4. **Waiting:** Thread waits for another thread's action.

5. **Timed Waiting:** Thread waits for a specified time.
6. **Terminated:** Thread completes execution.



Note: Running is a practical execution condition within the Runnable state in Java's official Thread.State enumeration.

4.6 Important Thread Methods

Method	Purpose
start()	Starts a new thread
run()	Contains thread execution code
sleep()	Pauses a thread temporarily
join()	Waits for another thread to finish

isAlive()	Checks whether a thread is active
setPriority()	Sets thread priority

Calling run() directly executes it like a normal method. It does not create a new thread. Calling start() creates a new thread and invokes run().

4.7 Behavioural Experiments

Run the program ten times and record the order in which scanner stations finish.

Run	Observed Finish Order
1	Station 2, 1, 4, 3
2	Station 1, 3, 2, 4
3	Station 4, 2, 1, 3

The actual order may differ between runs because of thread scheduling.

Thread priority does not guarantee execution order. A daemon thread runs in the background and may stop when all user threads finish.

4.8 Joint Conclusion

The same multithreaded program may produce different output orders because the operating system and JVM scheduler decide which thread executes first. A correct program should not depend on a particular execution order.

The join() method ensures that the main thread waits until all scanner stations complete before printing the final total.

4.9 Alternative Concept and Complexity Analysis

Instead of manually creating four Thread objects, ExecutorService can manage a fixed thread pool.

```
import java.util.concurrent.*;


public class ExecutorDemo {
  public static void main(String[] args) {
    ExecutorService pool = Executors.newFixedThreadPool(4);


    for (int i = 1; i <= 4; i++) {
```

```
        final int station = i;

        pool.submit(() ->
            System.out.println("Station " + station + " completed")
        );
    }

    pool.shutdown();
  }
}
```

For n scripts and p threads:

- Expected processing time is approximately O(n/p), assuming similar workloads and ignoring overhead.
- Creating one thread per station requires separate thread resources.
- A fixed thread pool limits the number of active threads to p.
- Thread stack memory is approximately O(p × stack size) for a pool of p threads.

4.10 Conclusion

Multithreading allows multiple scanner stations to process answer scripts simultaneously. Runnable provides flexibility, while ExecutorService simplifies thread management and improves resource utilization.

Q5 – RACE CONDITIONS IN A CINEMA SEAT-BOOKING SYSTEM

5.1 Problem Analysis

A cinema booking application allows multiple users to book seats simultaneously. If two users try to book the same seat at the same time, a race condition may occur.

A race condition happens when multiple threads access shared data simultaneously and produce an incorrect result.

5.2 Java Implementation Without Synchronization

```
public class SeatBooking {
  private final boolean[] booked;
```

```
private int bookingsMade = 0;

SeatBooking(int seats) {
   booked = new boolean[seats];
}

public void book(int seat) {
   if (!booked[seat]) {
      booked[seat] = true;
      bookingsMade++;
   }
}

public static void main(String[] args)
      throws InterruptedException {

   SeatBooking screen = new SeatBooking(10);
   Thread[] users = new Thread[100];

   for (int i = 0; i < users.length; i++) {
      final int seat = i % 10;
      users[i] = new Thread(() -> screen.book(seat));
      users[i].start();
   }

   for (Thread t : users)
      t.join();
```

```
    System.out.println("Seats sold: " + screen.bookingsMade);
  }
}
```

5.3 Problem with Unsynchronized Code

Without synchronization, multiple threads may check the same seat simultaneously before it is marked as booked. This can cause the same seat to be sold more than once or the booking count to become incorrect.

5.4 Correct Java Implementation Using Synchronization

```
public class SeatBooking {
  private final boolean[] booked;
  private int bookingsMade = 0;

  SeatBooking(int seats) {
    booked = new boolean[seats];
  }

  public synchronized void book(int seat) {
    if (!booked[seat]) {
      booked[seat] = true;
      bookingsMade++;
    }
  }

  public static void main(String[] args)
      throws InterruptedException {

    SeatBooking screen = new SeatBooking(10);
    Thread[] users = new Thread[100];
```

```
        for (int i = 0; i < users.length; i++) {
            final int seat = i % 10;
            users[i] = new Thread(() -> screen.book(seat));
            users[i].start();
        }

        for (Thread t : users)
            t.join();

        System.out.println("Seats sold: " + screen.bookingsMade);
    }
}
```

5.5 Expected Output

Seats sold: 10

5.6 Synchronization Methods

1. **Synchronized Method:** Locks the entire method so only one thread can execute it at a time on the same object.
2. **Synchronized Block:** Locks only the critical section of code, allowing better control over synchronization.
3. **Static Synchronization:** Locks the class object and is useful when multiple instances share static booking data.

The critical section is the code that checks whether a seat is available, marks it as booked and increments the booking count.

5.7 Performance Comparison

Method	Advantages	Disadvantages
Without synchronization	Faster execution in some cases	Race conditions may occur
Synchronized method	Prevents double booking	Locks the entire method

Synchronized block	Locks only required code	Requires careful implementation

Student A should run the unsynchronized version ten times and record any double bookings. Student B should measure the execution time of 10,000 booking attempts using each version.

5.8 Joint Engineering Decision

For a cinema with 20 screens, synchronized blocks are suitable because they can lock only the required seat-booking operation. Separate locks for individual screens can allow bookings on different screens to happen simultaneously.

5.9 Alternative Concept and Complexity Analysis

Other alternatives include:

- **AtomicBoolean:** Uses compareAndSet() to atomically change a seat from available to booked.
- **ReentrantLock:** Provides explicit locking and unlocking.
- **ConcurrentHashMap:** Uses putIfAbsent() to prevent duplicate seat entries.

Method	Booking Operation	Space Complexity
Synchronized array	O(1)	O(s)
AtomicBoolean array	O(1)	O(s)
ReentrantLock array	O(1)	O(s)
ConcurrentHashMap	O(1) average	O(s)

Here, s represents the number of seats. Under heavy contention, waiting time may increase because multiple users compete for the same seat.

5.10 Conclusion

Synchronization prevents race conditions and ensures that each cinema seat is booked only once. It maintains data consistency and provides reliable seat-booking operations in a multithreaded environment.

Q6 – DEADLOCK AND INTER-THREAD COMMUNICATION

6.1 Problem Analysis

This question contains two parts:

- **Part A – Deadlock:** Students transfer money between hostel wallet accounts. If two threads lock accounts in different orders, deadlock may occur.
- **Part B – Producer–Consumer:** A mess counter produces meal tokens, and students collect them. The counter can hold a maximum of five tokens.

6.2 Part A – Deadlock

Deadlock occurs when two or more threads wait permanently for resources held by each other.

Four Conditions of Deadlock

1. Mutual Exclusion – Only one thread can access a locked resource.
2. Hold and Wait – A thread holds one lock while waiting for another.
3. No Preemption – A lock cannot be forcibly taken away.
4. Circular Wait – Threads wait for resources held by each other.

Deadlock Prevention

Deadlock can be prevented by acquiring account locks in a consistent order, such as locking the account with the smaller account ID first.

This prevents circular waiting between threads.

6.3 Part B – Producer–Consumer Problem

The producer adds tokens to the counter, while consumers remove tokens.

- wait() makes a thread wait until a condition is satisfied.
- notify() wakes one waiting thread.
- notifyAll() wakes all waiting threads.
- sleep() pauses a thread for a specified time without releasing its lock.

The wait() method must be used inside a synchronized method or block and inside a loop to recheck the condition after waking.

6.4 Java Implementation

```
public class TokenCounter {

  private final int[] buffer = new int[5];

  private int count = 0, in = 0, out = 0;
```

```
public synchronized void put(int token)
      throws InterruptedException {

   while (count == buffer.length) {
      wait();
   }

   buffer[in] = token;
   in = (in + 1) % buffer.length;
   count++;

   notifyAll();
}

public synchronized int take()
      throws InterruptedException {

   while (count == 0) {
      wait();
   }

   int token = buffer[out];
   out = (out + 1) % buffer.length;
   count--;

   notifyAll();
   return token;
}
```

```
public static void main(String[] args)
    throws InterruptedException {

  TokenCounter counter = new TokenCounter();
  int[] collected = new int[2];

  Thread cook = new Thread(() -> {
    try {
      for (int t = 1; t <= 20; t++)
        counter.put(t);
    } catch (InterruptedException e) {
      Thread.currentThread().interrupt();
    }
  });

  Thread[] students = new Thread[2];

  for (int s = 0; s < 2; s++) {
    final int id = s;

    students[s] = new Thread(() -> {
      try {
        for (int k = 0; k < 10; k++)
          collected[id] += counter.take();
      } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
      }
```

```
        });
    }

    cook.start();

    for (Thread t : students)
        t.start();

    cook.join();

    for (Thread t : students)
        t.join();

    System.out.println("Sum of tokens collected: "
        + (collected[0] + collected[1])
        + " (expected 210)");
  }
}
```

6.5 Expected Output

Sum of tokens collected: 210 (expected 210)

6.6 Importance of wait() and notifyAll()

The while loop checks whether the counter is full or empty before performing an operation.

If if is used instead of while, a thread may continue even when the condition is no longer valid after waking up. This can cause incorrect results.

Using notifyAll() wakes all waiting threads so they can check the condition again. It is useful when multiple consumers and producers are waiting.

6.7 Test Cases

Test Case	Expected Result
Producer adds tokens	Tokens are added
Counter becomes full	Producer waits
Consumer removes token	Token is collected
Counter becomes empty	Consumer waits
Multiple consumers	Tokens are shared correctly

The counter size must always remain between 0 and 5.

6.8 Joint Conclusion

Deadlock and producer–consumer problems involve coordination between multiple threads. Consistent lock ordering prevents deadlock, while wait() and notifyAll() coordinate producers and consumers and maintain correct counter operations.

6.9 Alternative Concept and Complexity Analysis

Instead of wait() and notifyAll(), an ArrayBlockingQueue with capacity 5 can be used.

```
import java.util.concurrent.*;


public class QueueDemo {
  public static void main(String[] args)
      throws InterruptedException {


    BlockingQueue<Integer> queue =
      new ArrayBlockingQueue<>(5);


    for (int i = 1; i <= 5; i++)
      queue.put(i);


    System.out.println("Tokens: " + queue);


    System.out.println("Collected: " + queue.take());
```

```
    }
}
```

For wallet transfers, ReentrantLock with tryLock() and a timeout can be used. If a lock cannot be acquired within the timeout, the thread releases the lock it already holds and retries later.

Operation	Time Complexity
Add token	O(1)
Remove token	O(1)
Wallet transfer	O(1) for the balance update
Lock acquisition	Depends on contention

The token buffer uses O(5) = O(1) space. An ArrayBlockingQueue is generally less error-prone because it handles blocking and thread coordination internally.

6.10 Conclusion

Deadlock prevention and inter-thread communication are important for reliable multithreaded applications. Consistent lock ordering and ArrayBlockingQueue help improve safety and simplify thread coordination.